

Definir y llamar herramientas en la API

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 10 de 12 · Notas en español (es-ES)

El cuaderno en vivo recorrió una función **get-current-date-time** desde la definición hasta la frase final. Claude no conoce la hora de pared. Vuestra app sí. La API Messages es el contrato que permite al modelo *pedir* ese reloj sin fingir que ya lo tiene.

Dos artefactos que siempre escribís

1. La función

Una función normal en el lenguaje anfitrión. En la demo formateaba el «ahora» según una cadena de formato. Si nadie la llama con argumentos válidos, no aporta nada. Si alguien lo hace, devuelve datos del entorno de la *aplicación*, no de los pesos del modelo.

2. El esquema de la herramienta

El cliente de Anthropic espera un esquema que el modelo pueda leer. El formador insistió primero en dos campos:

- **name** — qué método invocar. Mantenedlo estable y único. Los nombres sobrecargados confunden al modelo.
- **description** — *cuándo* invocarla. Es la capa educativa. «Devuelve la fecha y hora actuales según la cadena de formato» basta para un reloj; un vago «obtiene datos» no.
- **esquema de entrada** — JSON Schema de parámetros (formato, ciudad, id de empleado). Si la función no toma argumentos, el esquema sigue existiendo; es vacío o mínimo.

Luego pasáis esos esquemas en el array `tools` de la llamada Messages, junto con el contenido de usuario. El enlace es por petición: podéis adjuntar una herramienta en este turno y cinco en el siguiente.

FIAROS DE LA DOCUMENTACIÓN VIGENTE

Los nombres de propiedades (`input_schema`, tipos de bloque, flags beta) hay que comprobarlos en **tool use**. Las capturas de agosto de 2026 no son una especificación.

El apretón de manos en un turno

1. Usuario: «¿Cuál es la hora exacta con formato HH:MM:SS?»
2. La petición incluye ese texto más el esquema de la herramienta reloj.
3. Claude **para** con motivo `tool_use`. No ha terminado la respuesta para el usuario.
4. El mensaje del asistente contiene un bloque **tool_use**: id, nombre `get_current_date_time` y la entrada elegida por el modelo (la cadena de formato).

5. Vuestro código mapea nombre → función, la ejecuta y obtiene un resultado como 7 : 29 : 31.
6. Añadís un mensaje de **usuario** cuyo contenido es un bloque **tool_result** (mismo id, la cadena resultado, `is_error` opcional).
7. Una llamada Messages de seguimiento deja que Claude escriba: «La hora exacta es 7:29:31».

Los roles de conversación siguen igual: system, user, assistant. Los resultados de herramientas viajan como contenido de *usuario* con un tipo de bloque especializado para que el modelo los una al id que emitió.

Cómo elige Claude entre varias herramientas

Lalit preguntó: si hay tres herramientas en el array, ¿quién elige? **El modelo**, usando nombres y descripciones (y el texto del usuario). No escribís un enrutador en la primera petición. Escribís mejores descripciones. Si la herramienta A devuelve errores que deben disparar la B, poned esa política en el **prompt de sistema** y/o en la descripción de B para que el siguiente turno vea el error y continúe.

Arvind preguntó si hay que adjuntar cinco herramientas siempre. No. Si conocéis la intención, adjuntad el subconjunto. Adjuntarlas todas es legal; las descripciones solapadas son el modo de fallo.

Una herramienta muchas veces, o varias en secuencia

El cuaderno mostró un bucle `while`: seguir llamando a la API hasta que el motivo de parada no sea `tool_use`. Dentro, recogéis cada bloque `tool_use`, ejecutáis funciones, empaquetáis `tool_result` y continuáis.

- **La misma herramienta dos veces en paralelo:** «Dadme la hora en HH:MM y también en formato de segundos». Dos bloques `tool_use`, dos cadenas de formato, una ronda de ejecución y luego una síntesis.
- **Herramientas dependientes:** «¿Qué hora será dentro de 90 minutos?» Primero `get_current_date_time`, luego `add_duration_to_date_time` con el reloj devuelto más 90 minutos. La segunda llamada necesita el primer resultado: secuencial, no paralela.
- **Despacho fijo:** `if/else` de aula según el nombre. Los marcos de producción suelen usar reflexión o un registro. En cualquier caso, *vuestro* proceso ejecuta la función.

Podéis cortar en código de aplicación (inspeccionar `is_error`, saltaros el modelo, llamar a un respaldo). El valor por defecto relevante para el examen, según Vijay, es: dejad planificar a Claude salvo que tengáis una rama barata y determinista. Más saltos al LLM significan más superficie de alucinación cuando la lista de herramientas crece.

Detalles de esquema que salieron en las preguntas

- Si queréis una sola llamada con muchos formatos, describid un **array** de cadenas de formato en lugar de depender de dos llamadas paralelas.
- Los nombres pueden ser `Clase.método` si vuestro despachador lo entiende. En las demos Python, un nombre de función plano era más simple; el estilo de clases en TypeScript vale si el mapeo es explícito.

- **tool_choice** (incluidas variantes de streaming/beta) puede forzar o sesgar qué herramienta se usa. Vijay señaló la vía beta de streaming como útil para cargas grandes y dijo que *no* era tema de examen en agosto de 2026. Confirmad la página vigente de la API.

Qué cambia el Agent SDK

El bucle manual es el concepto que importa en Foundation. En entrega a menudo llamaréis al Claude Agent SDK para no escribir vosotros el `while True`. El modelo sigue emitiendo `tool_use`; algo sigue ejecutando; los resultados siguen volviendo. El SDK es orquestación, no otra física.

POSTURA DE EXAMEN

Si un escenario describe el motivo de parada `tool_use`, el siguiente paso correcto es: ejecutar la herramienta en local (o vía MCP), devolver `tool_result`, volver a llamar a la API — no inventar el reloj en el texto del asistente.

Documentación oficial

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Esquemas, motivos de parada, `tool_result`.

- **API Messages**

<https://docs.claude.com/en/api/messages>

Parámetro `tools` en cada petición.

- **Streaming**

<https://docs.claude.com/en/docs/build-with-claude/streaming>

Si combináis herramientas con tokens en streaming; comprobad el `tool_choice` actual.

- **Salidas estructuradas**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Relacionado: JSON que el modelo debe emitir frente a JSON que devuelve una herramienta.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.